Course: Introduction to AI

Prof. Ahmed Guessoum
The National Higher School of AI

Chapter 7

Logical Agents

Outline

- Knowledge-Based Agents
- The Wumpus World
- Logic
- Propositional Logic: A Very Simple Logic
  - Syntax
  - Semantics
  - A Simple Knowledge Base
  - A Simple Inference Procedure
- Propositional Theorem Proving
  - Inference and Proofs
  - Proof by Resolution
  - Horn Clauses and Definite Clauses
  - Forward and Backward Chaining

Outline

- Effective Propositional Model Checking
  - A Complete Backtracking Algorithm
  - Local Search Algorithms
  - The Landscape of Random Sat Problems
- Agents Based on Propositional Logic
  - The Current State of the World
  - A Hybrid Agent
  - Logical State Estimation
  - Making Plans by Propositional Inference

On Knowledge Representation Power

- Humans have an internal **representation of knowledge** and they **reason** with this knowledge.
- In AI, this approach to intelligence is embodied in **knowledge-based agents**.
- The agents seen in chap. 3 & 4 allow to move from one state to another using domain-specific function, but do not allow to reason more generally about the states. E.g. in 8-puzzle that:
  - two tiles cannot occupy the same space
- Also, an agent's only choice for representing what it knows about the current state is to list all possible concrete states—hopeless in large environments

On Knowledge Representation Power

- CSPs represent states as assignments of values to variables ➔ some parts of the agent work in a domain-independent way, allowing for more efficient algorithms.
- In this chapter (and following ones) logic is presented as a general class of representations to support knowledge-based agents.
- Combination of Knowledge allowing for reasoning (inference).
- Knowledge-based agents can:
  - accept new tasks in the form of explicitly described goals;
  - achieve competence quickly by being told or learning new knowledge about the environment; and
  - adapt to changes in the environment by updating the relevant knowledge.

Knowledge-Based Agents

- The central component of a knowledge-based agent is its **knowledge base** (KB).
- A knowledge base is a set of **sentences** expressed in a language called a **knowledge representation language.**
- **Axiom**: a sentence taken as given without being derived from other sentences.
- *TELL*: a way to add new sentences to the KB and
- *ASK*: a way to query the KB.
- Both operations may involve deriving new sentences from old ones.
- The agent *maintains* a knowledge base, KB, which may initially contain some **background knowledge**.

A Generic Knowledge-Based Agent

```
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
              t, a counter, initially 0, indicating time

  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Figure 7.1 A generic knowledge-based agent. Given a percept, the agent adds the percept to its knowledge base, asks the knowledge base for the best action, and tells the knowledge base that it has in fact taken that action.

- When agent program called:
  - TELLs the KB what it perceives.
  - ASKs the KB what action it should perform (after reasoning about the current state of the world, about the outcomes of possible action sequences, etc.).
  - TELLs the KB which action was chosen, and the agent executes it.
- The details of the representation language are hidden in the 3 functions.
- The details of inference mechanisms hidden in *TELL* and *ASK.*

Declarative vs Procedural K.

- We will mostly be concerned with stating the knowledge that the agent needs to reason about its environment.
- The agent will add more and more such knowledge until it knows how to operate in its environment.
- All this is done at the **knowledge level**; we will refer to this as a **declarative** approach to system building.
- The **implementation level** is not our primary concern: e.g.
  - Is the geographical knowledge implemented as linked lists or pixel maps?
  - Does the agent reason using strings and symbols stored in registers or by propagating noisy signals in a network of neurons?
- The **procedural** approach encodes desired behaviours directly as program code.

The Wumpus World

- The **wumpus world** is a cave consisting of rooms connected by passageways.
- Somewhere in the cave is the terrible wumpus, a beast that eats anyone who enters its room.
- The wumpus can be shot by an agent, but the agent has only one arrow.
- Some rooms contain bottomless pits that will trap anyone who wanders into these rooms (except for the wumpus, which is too big to fall in).
- There is a possibility of finding a heap of gold.
- Although the wumpus world is rather tame by modern computer game standards, it illustrates some important points about intelligence.

PEAS description of Wumpus World

- **Performance measure**:
  - +1000 for climbing out of the cave with the gold,
  - –1000 for falling into a pit or being eaten by the wumpus,
  - –1 for each action taken and
  - –10 for using up the arrow.
  - The game ends either when the agent dies or when the agent climbs out of the cave.
- **Environment**: A 4×4 grid of rooms.
  - The agent always starts in the square labeled [1,1], facing to the right.
  - The locations of the gold and the wumpus are chosen randomly, with a uniform distribution, from the squares other than the start square.
  - Each square other than the start can be a pit, with probability 0.2.

The Wumpus World

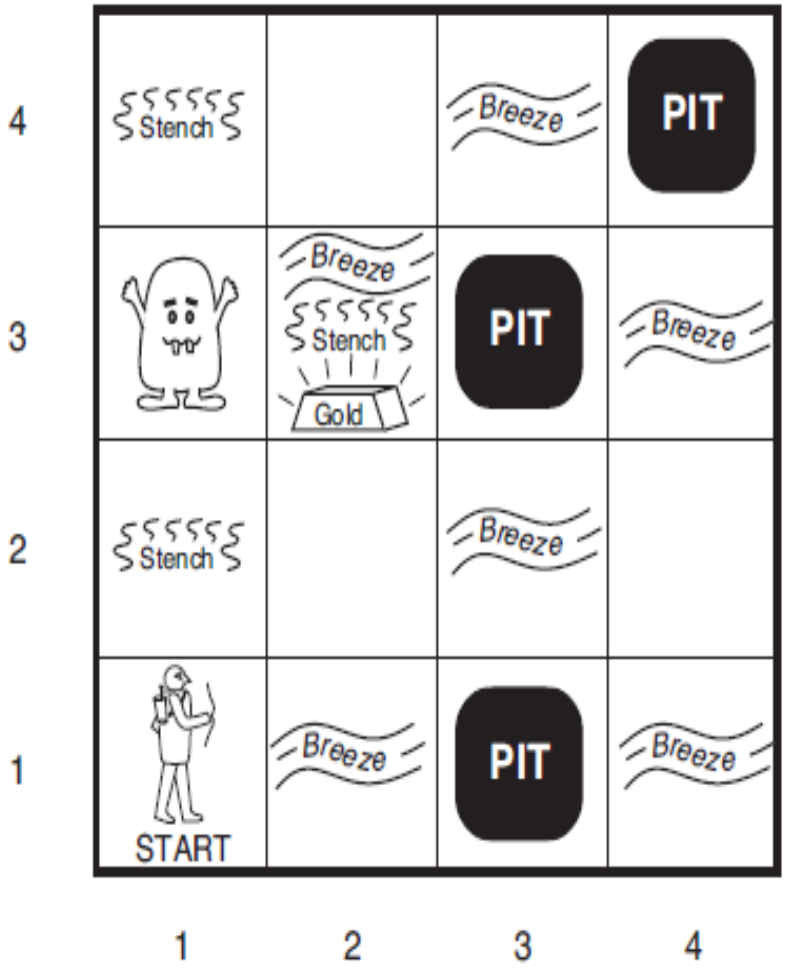


Figure 7.2 A typical wumpus world. The agent is in the bottom left corner, facing right.

PEAS description of Wumpus World

- **Actuators**:
  - Agent can move *Forward*, *TurnLeft* by 90◦, or *TurnRight* by 90◦.
  - Agent dies if it enters a square containing a pit or a live wumpus. (It is safe, although smelly, to enter a square with a dead wumpus.)
  - If agent tries to move forward and bumps into a wall, then the agent does not move.
  - The action *Grab* can be used to pick up the gold if it is in the same square as the agent.
  - The action *Shoot* can be used to fire an arrow in a straight line in the direction the agent is facing. The arrow continues until it either hits (and hence kills) the wumpus or hits a wall.
  - The agent has only one arrow, so only the first *Shoot* action has any effect.
  - Finally, the action *Climb* can be used to climb out of the cave, but only from square [1,1].

PEAS description of Wumpus World

- **Sensors**: The agent has five sensors, each of which gives a single, different information:
  - In the square containing the wumpus and in the directly (not diagonally) adjacent squares, the agent will perceive a *Stench*.
  - In the squares directly adjacent to a pit, the agent will perceive a *Breeze*.
  - In the square where the gold is, the agent will perceive a *Glitter*.
  - When an agent walks into a wall, it will perceive a *Bump*.
  - When the wumpus is killed, it emits a horrible *Scream* that can be perceived anywhere in the cave.
- The percepts will be given to the agent program in the form of a list of five symbols;

E.g., if there is a *stench* and a *breeze*, but no *glitter*, *bump*, or *scream*, the agent will get [Stench, Breeze, None, None, None].

The Wumpus World: reasoning

- Main challenge for agent in the environment:
  its initial ignorance of the configuration of the environment
  ➔ logical reasoning is required to overcome it.
- Informal knowledge representation language used: writing down symbols in a grid.
- The agent's initial KB contains the rules of the environment, as described previously;
  - In particular, it knows that it is in [1,1] and that [1,1] is a safe square; we denote that with an "A" and "OK," respectively, in square [1,1].
  - The first percept is [None, None, None, None, None], so the agent can conclude that its neighbouring squares, [1,2] and [2,1], are free of dangers—they are OK.
- The reasoning continues…

The Wumpus World: Representation & Reasoning

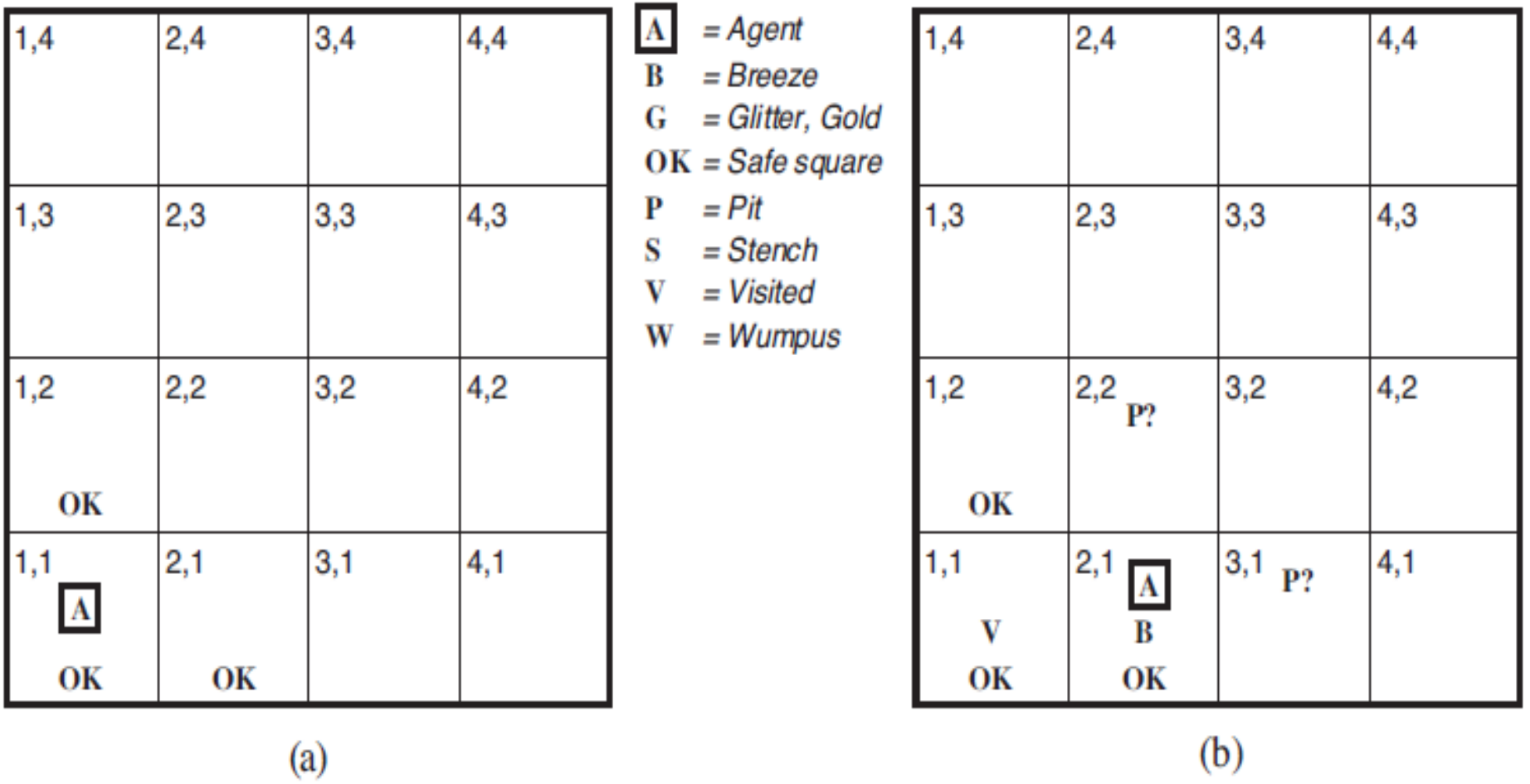


Figure 7.3 The first step taken by the agent in the wumpus world. (a) The initial situation, after percept [*None*, *None*, *None*, *None*, *None*]. (b) After one move, with percept [*None*, *Breeze*, *None*, *None*, *None*].

The Wumpus World: Representation & Reasoning

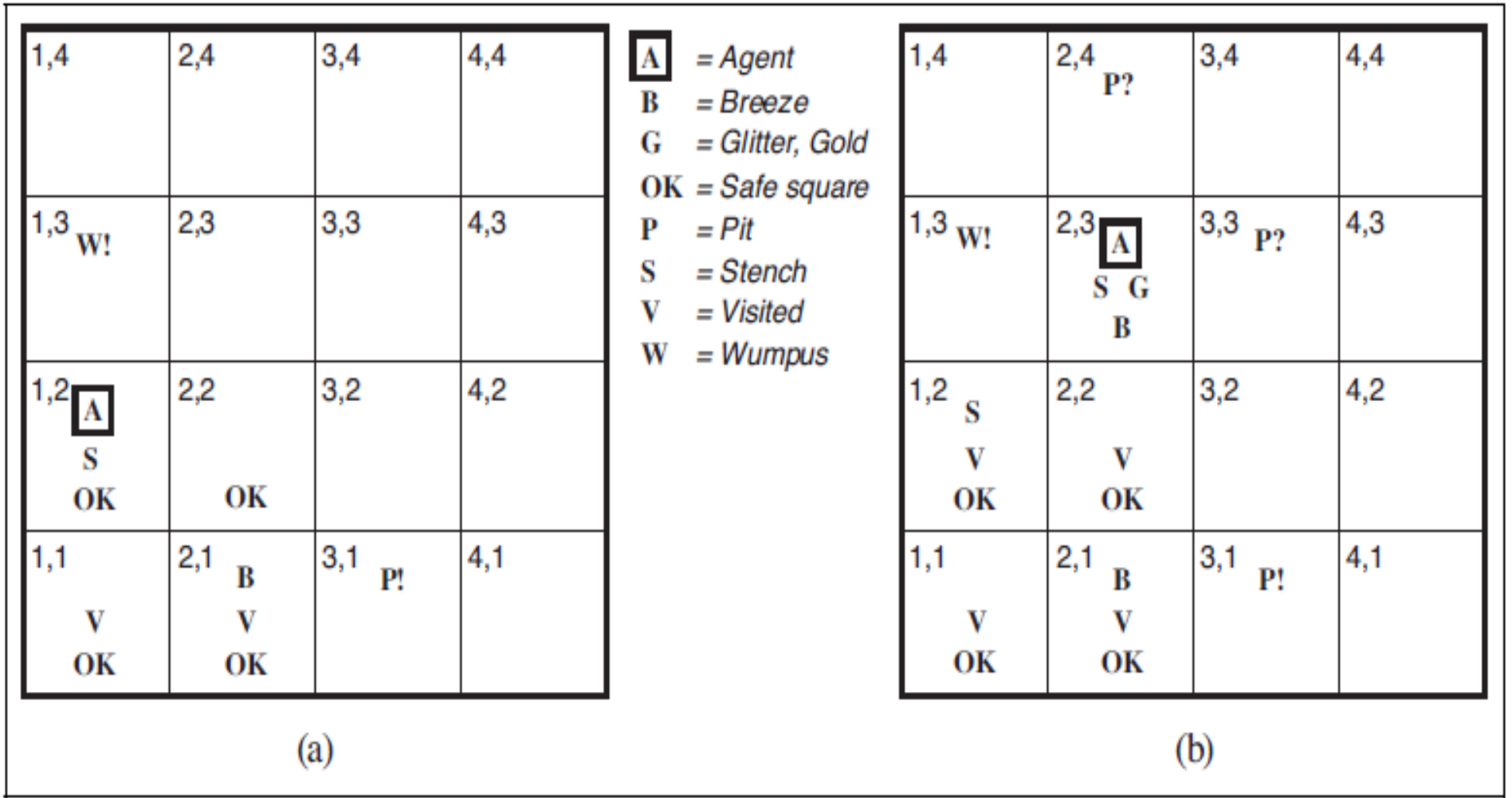


Figure 7.4 Two later stages in the progress of the agent. (a) After the third move, with percept [*Stench*, *None*, *None*, *None*, *None*]. (b) After the fifth move, with percept [*Stench*, *Breeze*, *Glitter*, *None*, *None*].

Outline

- Knowledge-Based Agents
- The Wumpus World
- Logic
- Propositional Logic: A Very Simple Logic
  - Syntax
  - Semantics
  - A Simple Knowledge Base
  - A Simple Inference Procedure
- Propositional Theorem Proving
  - Inference and Proofs
  - Proof by Resolution
  - Horn Clauses and Definite Clauses
  - Forward and Backward Chaining

Logic

- Recall that KBs consist of sentences.
- Sentences are expressed according to the **syntax** of the representation language, which specifies all the sentences that are well formed.
  - Example: for the language of *arithmetic* "x + y = 4" is a well-formed sentence, whereas "x4y+ =" is not.
- A logic must also define the **semantics** or meaning of sentences.
- The semantics defines the **truth** of each sentence with respect to each **possible world**.
  - E.g. the semantics for arithmetic specifies that the sentence "x + y = 4" is true in a world where x is 2 and y is 2, but false in a world where x is 1 and y is 5.
- In standard logics, every sentence must be either true or false in each possible world. (Fuzzy logic is different.)

Logic

- To be formal (and avoid a looser usage of the word "model" in your textbook) we will adopt the following definitions and reflect them in the sequel:
  - A "possible world" (or **interpretation**) is an assignment that fixes the truth of any sentence as **true** or **false**.
  - A **model** is an interpretation that makes a sentence **true**.
- E.g. for sentence "$x+y=4$", the possible models are all possible assignments of real numbers to x and y that make the sentence **true**.
- If a sentence α is true in model m, we say that m **satisfies** α or sometimes m **is a model of** α.
- The notation $M(\alpha)$ is used to mean the set of all models of α.
- Logical **entailment** between sentences: $\alpha \models \beta$ states that sentence α entails sentence β (i.e. β logically follows from α).

Logical Entailment

- Formal definition of entailment:

  $\alpha \models \beta$ if and only if, for every model of α, β is also true.
- This can be written as: $\alpha \models \beta$ if and only if $M(\alpha) \subseteq M(\beta)$. (Note the direction of the $\subseteq$ here: if $\alpha \models \beta$, then α is a s*tronger* assertion than β: it rules out *more* possible worlds.)
  - E.g. $(x = 0) \models (xy = 0)$.
- Wumpus World:
  - The agent has detected nothing in [1,1] and a breeze in [2,1].
  - At this point, KB = union of these percepts and the agent's knowledge of the rules of the wumpus world.
  - Do the adjacent squares [1,2], [2,2], and [3,1] contain pits?
  - There are $2^3 = 8$ possible models for these squares.

Logical Entailment

- The KB is false in interpretations that contradict Agent's knowledge
  - E.g., the KB is false in any interpretation in which [1,2] contains a pit, because there is no breeze in [1,1].
  - There are in fact just three models of KB (solid line in figure below).
- Consider two possible conclusions:

  α_1 = "There is no pit in [1,2]"

  α_2 = "There is no pit in [2,2]"
- By inspection, we see that for every model of *KB*, α_1 is also true.

  Hence, $KB \models \alpha_1$: there is no pit in [1,2].
- We can also see that in some models of *KB*, α_2 is false.

  Hence, $KB \not\models \alpha_2$: the agent *cannot* conclude that there is no pit in [2,2]. (Nor can it conclude that there *is* a pit in [2,2].)

Wumpus World & Possible Worlds

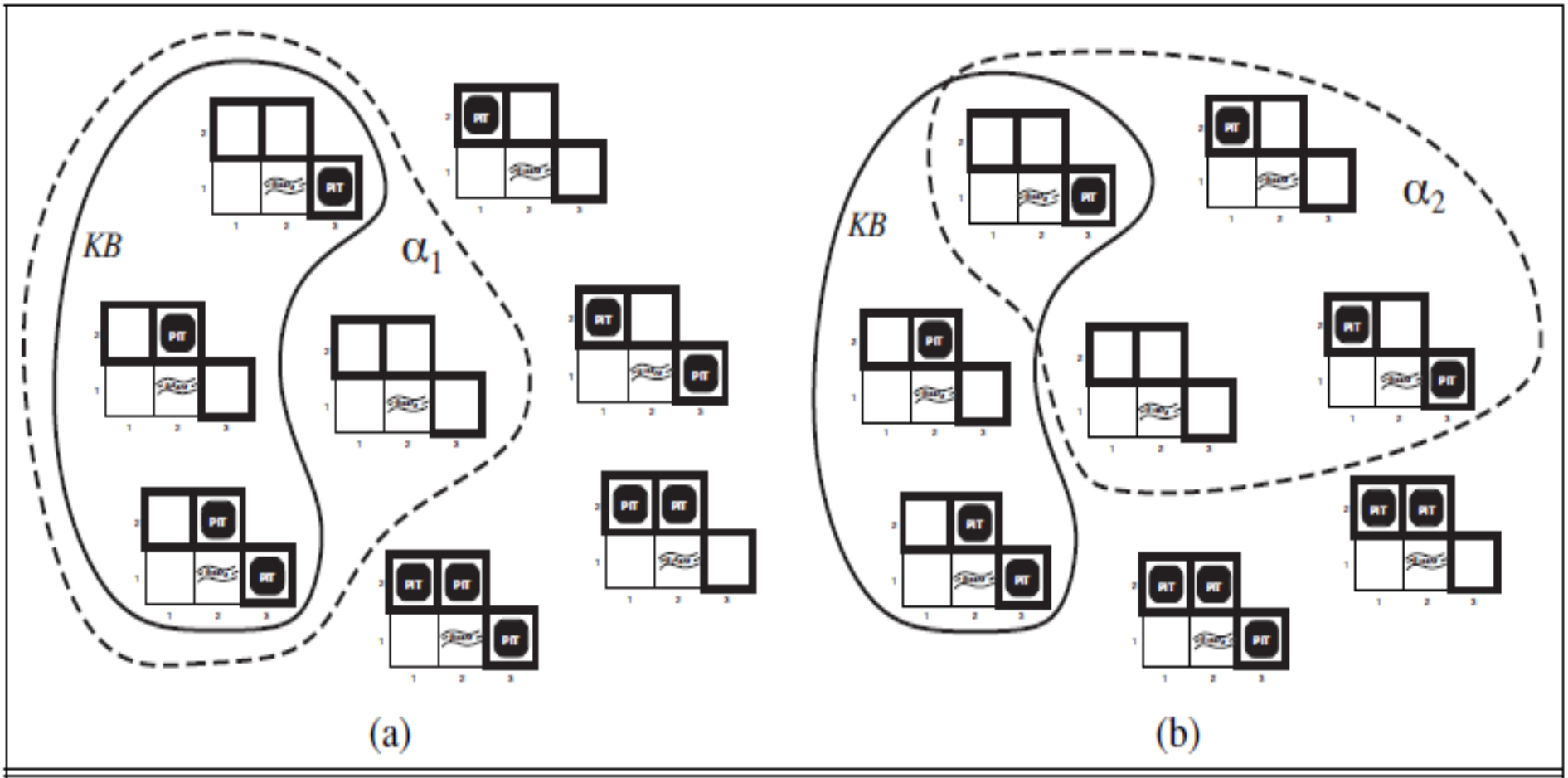


Figure 7.5 Possible models for the presence of pits in squares [1,2], [2,2], and [3,1]. The KB corresponding to the observations of nothing in [1,1] and a breeze in [2,1] is shown by the solid line. (a) Dotted line shows models of α_1 (no pit in [1,2]). (b) Dotted line shows models of α_2 (no pit in [2,2]).

Logical Entailment and Inference

- The preceding example
  - illustrates entailment, and
  - shows how the definition of entailment can be applied to derive conclusions—i.e. to carry out **logical inference**.
- **Model checking**: enumerating all possible interpretations to check that α is true in all models of KB, i.e. that M(KB) $\subseteq$ M(α).
- If an **inference algorithm** *i* can derive α from KB, we write $KB \vdash_i \alpha$. (We say "α is derived from *KB* by *i*")
- An inference algorithm that derives only entailed sentences is called **sound** (or **truth-preserving**).

i.e. *i* is sound if whenever $KB \vdash_i \alpha$ it is also true that $KB \models \alpha$

- An inference algorithm is **complete** if it can derive any sentence that is entailed.

i.e. *i* is complete if whenever $KB \models \alpha$ it is also true that $KB \vdash_i \alpha$

Propositional Logic: Syntax

- **Syntax** defines the allowable sentences in any language.
- An **atomic sentence** consists of a single **proposition symbol**. Each symbol stands for a proposition that can be true or false.
- Proposition symbols start with an uppercase letter. E.g., *P, Q, R,* $W_{1,3}$ and *North*.
- Proposition symbols with fixed meanings: **True** and **False**.
- **Complex sentences** are constructed from simpler sentences, using parentheses and **logical connectives**.
- There are five connectives in common use:
  - ¬ (not). ¬ $W_{1,3}$ is called the **negation** of $W_{1,3}$
  - A **literal** is either an atomic sentence (a **positive literal**) or a negated atomic sentence (a **negative literal**).

Propositional Logic: Syntax

- ∧ (and): A sentence whose main connective is ∧ is called a **conjunction**; its parts are the **conjuncts**.
- ∨ (or): A sentence using ∨ is a **disjunction;** its parts are the **disjuncts**

- ⇒ (implies): A sentence such as $(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$ is called an **implication** (or conditional). Its **premise** or **antecedent** is $(W_{1,3} \wedge P_{3,1})$, and its **conclusion** or **consequent** is $\neg W_{2,2}$.
  - Implications are also known as **rules** or **if–then** statements.
  - The implication symbol is sometimes written in other books as ⊃ or →.
- ⇔ (if and only if): A sentence connected by a ⇔ symbol is said to be a **biconditional**.

BNF of Propositional Logic

$$
\begin{aligned}
\textit{Sentence} &\rightarrow \textit{AtomicSentence} \mid \textit{ComplexSentence} \\
\textit{AtomicSentence} &\rightarrow \textit{True} \mid \textit{False} \mid P \mid Q \mid R \mid \ldots \\
\textit{ComplexSentence} &\rightarrow (\ \textit{Sentence}\ ) \mid [\ \textit{Sentence}\ ] \\
&\mid \neg\ \textit{Sentence} \\
&\mid \textit{Sentence} \wedge \textit{Sentence} \\
&\mid \textit{Sentence} \vee \textit{Sentence} \\
&\mid \textit{Sentence} \Rightarrow \textit{Sentence} \\
&\mid \textit{Sentence} \Leftrightarrow \textit{Sentence} \\
\text{OPERATOR PRECEDENCE} &: \neg, \wedge, \vee, \Rightarrow, \Leftrightarrow
\end{aligned}
$$

Figure 7.7 A BNF (Backus–Naur Form) grammar of sentences in propositional logic, along with operator precedences, from highest to lowest.

Logical operators have precedence from ¬ (highest), then ∧, then ∨, then ⇒, and finally ⇔ (lowest)

Propositional Logic: Semantics

- The semantics defines the rules for determining the truth of a sentence with respect to a particular model.
- In propositional logic, an interpretation simply fixes the **truth value** (true or false) for every proposition symbol.
  - If the sentences in the KB make use of the proposition symbols $P_{1,2}$, $P_{2,2}$, and $P_{3,1}$, then one possible interpretation for KB is $m_1 = \{P_{1,2}=\text{false}, P_{2,2}=\text{false}, P_{3,1}=\text{true}\}$.
- The semantics for propositional logic specifies how to compute the truth value of *any* sentence, given an intepretation. This is done recursively starting with the atomic sentences using a **truth table**.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
false	*false*	*true*	*false*	*false*	*true*	*true*
false	*true*	*true*	*false*	*true*	*true*	*false*
true	*false*	*false*	*false*	*true*	*false*	*false*
true	*true*	*false*	*true*	*true*	*true*	*true*

Figure 7.8 Truth tables for the five logical connectives. To use the table to compute, for

The Wumpus World

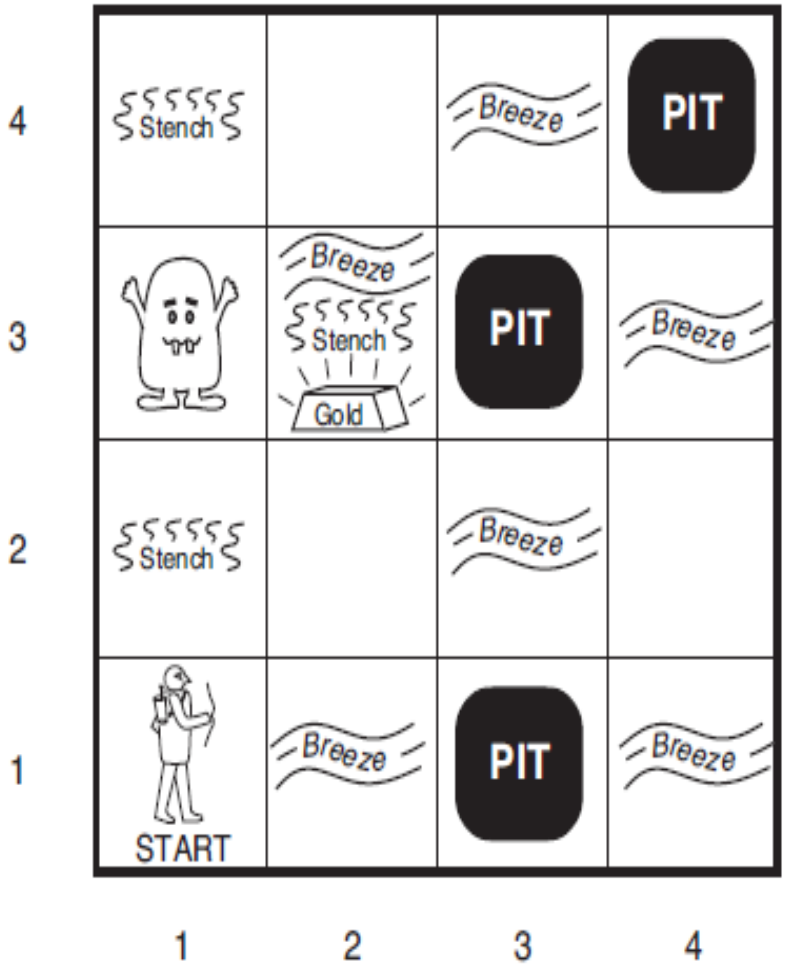


Figure 7.2 A typical wumpus world. The agent is in the bottom left corner, facing right.

The Wumpus World: Representation & Reasoning

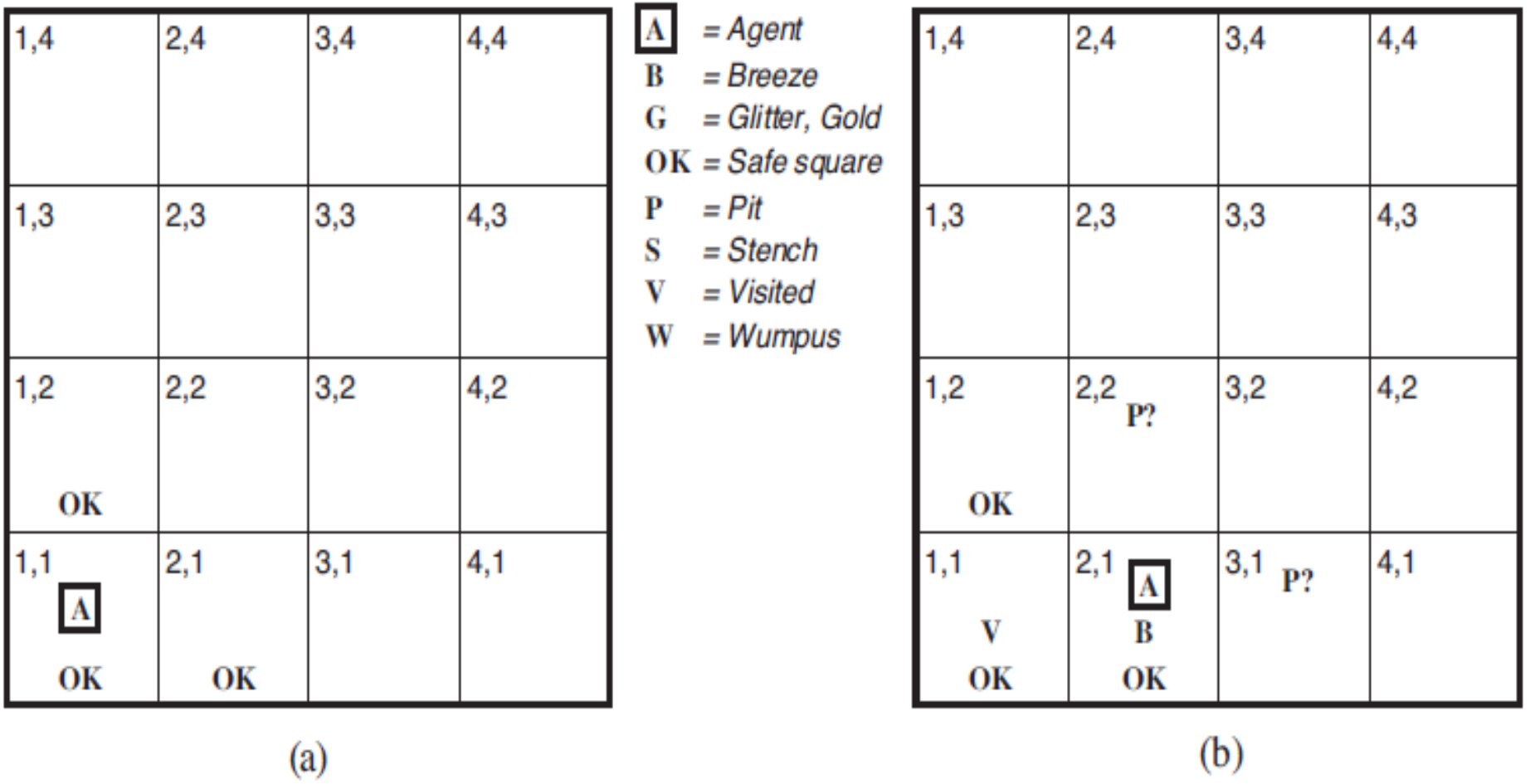


Figure 7.3 The first step taken by the agent in the wumpus world. (a) The initial situation, after percept [*None*, *None*, *None*, *None*, *None*]. (b) After one move, with percept [*None*, *Breeze*, *None*, *None*, *None*].

A simple knowledge base

A KB for the Wumpus World:

- Symbols for each [x,y] location:
  - $P_{x,y}$ is true if there is a pit in [x, y].
  - $W_{x,y}$ is true if there is a wumpus in [x, y], dead or alive.
  - $B_{x,y}$ is true if the agent perceives a breeze in [x, y].
  - $S_{x,y}$ is true if the agent perceives a stench in [x, y].
- Sentences: Each sentence R_i is labelled for referencing purposes.
- There is no pit in [1,1]: $R_1 : \neg P_{1,1}$
- A square is breezy if and only if there is a pit in a neighbouring square. To be stated for each square; (for now, just the relevant sentences are included):

  $R_2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ $R_3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- Adding the breeze percepts for the first two squares visited in the specific world the agent is in (see Figure 7.3(b))

  $R_4 : \neg B_{1,1}$ $R_5 : B_{2,1}$

A simple inference procedure

- The goal now is to decide whether KB |= α for some sentence α.
- **First inference algorithm**: model-checking approach (direct implementation of *entailment* definition): enumerate the interpretations, and check that α is true in every model of *KB*.
- In Wumpus world example:
  - 7 relevant proposition symbols (at some point of the game): $B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, and $P_{3,1}$
  - $2^7 = 128$ possible interpretations.
  - Three of these are models of KB.
  - In those 3 models, $\neg P_{1,2}$ (R_1) is true, hence there is no pit in [1,2].
  - But, $P_{2,2}$ is true in 2 of the 3 models and false in 1, so we cannot tell whether there is a pit in [2,2].

A simple inference procedure

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1	R_2	R_3	R_4	R_5	KB
false	*false*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*false*	*false*
false	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*false*	*true*	*false*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
false	*true*	*false*	*false*	*false*	*false*	*false*	*true*	*true*	*false*	*true*	*true*	*false*
false	*true*	*false*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*false*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*false*	*true*	*true*	*true*	*true*	*true*	*true*	*true*	*true*
false	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*false*	*false*	*true*	*true*	*false*
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮
true	*true*	*true*	*true*	*true*	*true*	*true*	*false*	*true*	*true*	*false*	*true*	*false*

Figure 7.9 A truth table constructed for the knowledge base given in the text. *KB* is true if R_1 through R_5 are true, which occurs in just 3 of the 128 rows (the ones underlined in the right-hand column). In all 3 rows, $P_{1,2}$ is false, so there is no pit in [1,2]. On the other hand, there might (or might not) be a pit in [2,2].

A truth-table enumeration algorithm for deciding propositional entailment

function TT-ENTAILS?(KB, α) **returns** *true* or *false*
  inputs: KB, the knowledge base, a sentence in propositional logic
    α, the query, a sentence in propositional logic

  symbols $\leftarrow$ a list of the proposition symbols in KB and α
  return TT-CHECK-ALL(KB, α, *symbols*, { })

---

function TT-CHECK-ALL(KB, α, *symbols*, *model*) **returns** *true* or *false*
  if EMPTY?(*symbols*) **then**
    if PL-TRUE?(KB, *model*) **then return** PL-TRUE?(α, *model*)
    else return *true* // *when KB is false, always return true*
  else do
    $P \leftarrow$ FIRST(*symbols*)
    rest $\leftarrow$ REST(*symbols*)
    return (TT-CHECK-ALL(KB, α, *rest*, *model* $\cup \{P = true\}$)
      and
      TT-CHECK-ALL(KB, α, *rest*, *model* $\cup \{P = false\}$))

Figure 7.10 A truth-table enumeration algorithm for deciding propositional entailment. (TT stands for truth table.) PL-TRUE? returns *true* if a sentence holds within a model. The variable *model* represents a partial model—an assignment to some of the symbols. The keyword "**and**" is used here as a logical operation on its two arguments, returning *true* or *false*.

Properties of TT-ENTAILS?

- The *TT-ENTAILS?* algorithm is
  - **Sound** because it implements directly the definition of entailment, and
  - **Complete** because it works for any KB and α and always terminates. (There are only finitely many interpretations to examine and it enumerates them)
- Complexity of *TT-ENTAILS?*
  - If KB and α contain n symbols in all, then there are 2^n interpretations. Thus, the time complexity of the algorithm is $O(2^n)$.
  - Space complexity: O(n) because the enumeration is depth-first.

Standard logical equivalences

$$
\begin{aligned}
(\alpha \wedge \beta) &\equiv (\beta \wedge \alpha) && \text{commutativity of } \wedge \\
(\alpha \vee \beta) &\equiv (\beta \vee \alpha) && \text{commutativity of } \vee \\
((\alpha \wedge \beta) \wedge \gamma) &\equiv (\alpha \wedge (\beta \wedge \gamma)) && \text{associativity of } \wedge \\
((\alpha \vee \beta) \vee \gamma) &\equiv (\alpha \vee (\beta \vee \gamma)) && \text{associativity of } \vee \\
\neg(\neg\alpha) &\equiv \alpha && \text{double-negation elimination} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\beta \Rightarrow \neg\alpha) && \text{contraposition} \\
(\alpha \Rightarrow \beta) &\equiv (\neg\alpha \vee \beta) && \text{implication elimination} \\
(\alpha \Leftrightarrow \beta) &\equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)) && \text{biconditional elimination} \\
\neg(\alpha \wedge \beta) &\equiv (\neg\alpha \vee \neg\beta) && \text{De Morgan} \\
\neg(\alpha \vee \beta) &\equiv (\neg\alpha \wedge \neg\beta) && \text{De Morgan} \\
(\alpha \wedge (\beta \vee \gamma)) &\equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma)) && \text{distributivity of } \wedge \text{ over } \vee \\
(\alpha \vee (\beta \wedge \gamma)) &\equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma)) && \text{distributivity of } \vee \text{ over } \wedge
\end{aligned}
$$

Figure 7.11 Standard logical equivalences. The symbols α, β, and γ stand for arbitrary sentences of propositional logic.

Propositional Theorem Proving

- We have seen how entailment can be shown by **model checking**: enumerating interpretations and showing that the sentence must hold in all models.
- We will show how entailment can be done by **theorem proving**: applying rules of inference directly to the KB sentences to construct a proof of a given sentence without consulting interpretations.
- If the number of interpretations is large but the length of the proof is short, then theorem proving can be more efficient than model checking.
- **logical equivalence**: two sentences α and β are logically equivalent (written $\alpha \equiv \beta$):
  - if they are true in the same set of models. Alternatively,
  - only if each of them entails the other:
    $\alpha \equiv \beta$ if and only if $\alpha \models \beta$ and $\beta \models \alpha$

Validity and Satisfiability

- A sentence is **valid** if it is true in *all* models. E.g., *True*, *P* ∨ ¬*P*, $A \Rightarrow A, (A \wedge (A \Rightarrow B)) \Rightarrow B$ *are all valid sentences*
- Valid sentences are also known as **tautologies**—they are *necessarily true*. (Every valid sentence is logically equivalent to *True*).
- Validity is connected to inference via the **deduction theorem**: KB |= α *iff* $(KB \Rightarrow \alpha)$ *is valid. (Proof as ex.)*
- A sentence is **satisfiable** if it is *true* in *some* model: $A \vee B$, C
- A sentence is **unsatisfiable** if it is true in *no* model: $A \wedge \neg A$
- In the wumpus KB, (R1 ∧ R2 ∧ R3 ∧ R4 ∧ R5), is satisfiable because there are three models in which it is true.
- Satisfiability can be checked by enumerating the possible models until one is found that satisfies the sentence

Satisfiability and proofs

- The problem of determining the satisfiability of sentences SAT in propositional logic, the **SAT** problem, was the first problem proved to be NP-complete.
- Many problems in CS are satisfiability problems. E.g., all the CSPs ask whether the constraints are satisfiable by some assignment.
- α is valid iff $\neg\alpha$ is unsatisfiable; contrapositively, α is satisfiable iff $\neg\alpha$ is not valid.
- Another useful result: $KB \models \alpha$ iff $(KB \wedge \neg \alpha)$ *is unsatisfiable.*
- Proving α from KB by checking the unsatisfiability of $(KB \wedge \neg \alpha)$ corresponds exactly to a proof by **contradiction** (or proof by **refutation)**.
- One assumes a sentence α to be false and shows that this leads to a contradiction with known axioms KB.

Inference and proofs

- Proof methods are of two types: (1) model checking or (2) application of inference rules.
- An **inference rule:** a rule that can be applied as one step in a **proof**— i.e. in a chain of conclusions that leads to the desired goal.
- Example of inference rule: **Modus Ponens**

$$\frac{\alpha \Rightarrow \beta \quad \alpha}{\beta}$$

  - *if (WumpusAhead* $\wedge$ *WumpusAlive)* $\Rightarrow$ *Shoot* and *WumpusAhead* and *WumpusAlive* are given, then Shoot can be inferred.

- Other inference rule: **And-Elimination** $\dfrac{\alpha \wedge \beta}{\beta}$

Inference and proofs

- By considering the possible truth values of α and β, one can show that *Modus Ponens* and *And-Elimination* are sound.
- All the logical equivalences in Figure 7.11 can be used as inference rules.
- One can use inference rules as operators in a standard search algorithm.
- The application of inference rules typically requires the translation of sentences into a normal form.

Inference rules and equivalences in the wumpus world:

- We start with the KB containing R_1 to R_5 and show how to prove $\neg P_{1,2}$ (there is no pit in [1,2]).
- First, we apply biconditional elimination to R_2 to obtain

  R_6: $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
- Then we apply And-Elimination to R_6 to obtain

  R_7: $(P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}$
- Logical equivalence for contrapositives gives

  R_8: $\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})$
- Apply Modus Ponens with R_8 and the percept R_4 ($\neg B_{1,1}$):

  gives R_9 : $\neg(P_{1,2} \vee P_{2,1})$
- Finally, apply De Morgan's rule, giving the conclusion

  R_{10} : $\neg P_{1,2} \wedge \neg P_{2,1}$
- That is, neither [1,2] nor [2,1] contains a pit.

Proofs as Search

- This proof was found by hand.
- Any of the search algorithms in Chapter 3 can be applied to find a sequence of steps that constitutes a proof.
- A proof problem can be defined as follows:
  - INITIAL STATE: the initial KB.
  - ACTIONS: the set of actions consists of all the inference rules applied to all the sentences that match the top half of the inference rule.
  - RESULT: the result of an action is to add the sentence in the bottom half of the inference rule to the KB.
  - GOAL: the goal is a state that contains the sentence we are trying to prove.
- Searching for proofs is alternative to enumerating models.
- Practically, *finding a proof can be more efficient because the proof can ignore irrelevant propositions.*

Monotonicity

- Final property of logical systems is **monotonicity**: the set of entailed sentences can only *increase* as information is added to the KB.
- For any sentences α and β, if KB |= α then KB ∧ β |= α
- Suppose our KB contains the additional assertion β that *there are exactly eight pits in the Wumpus world*.
- This knowledge might help the agent draw *additional* conclusions, but it cannot invalidate any conclusion α already inferred—such as the conclusion that there is no pit in [1,2].
- Monotonicity means that inference rules can be applied whenever suitable premises are found in the KB—the conclusion of the rule must follow *regardless of what else is in the KB*.

Proof by Resolution

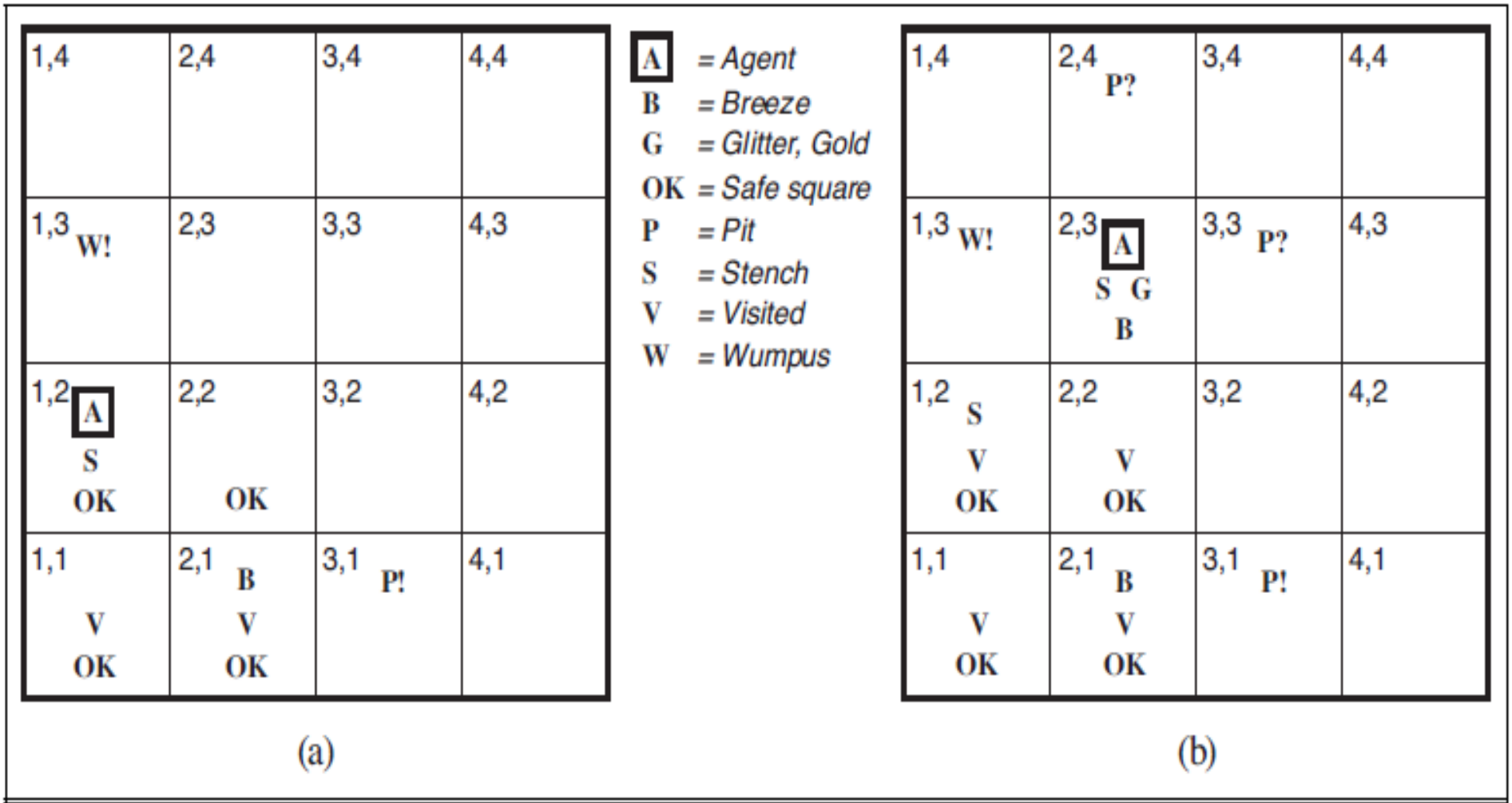


Figure 7.4 Two later stages in the progress of the agent. (a) After the third move, with percept [*Stench*, *None*, *None*, *None*, *None*]. (b) After the fifth move, with percept [*Stench*, *Breeze*, *Glitter*, *None*, *None*].

Proof by Resolution

- **Resolution**: a single inference rule that yields a complete inference algorithm when coupled with any complete search algorithm.
- Consider the agent reaching square [1,2] in Figure 7.4(a).
- Percepts: [Stench, None, None, None, None].
- ➔ We add the following facts (+ others) to the KB:
  R_{11}: $\neg B_{1,2}$ and R_{12}: $B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$
- From these 2 sentences we get:
  R_{13}: $\neg P_{2,2}$ and R_{14}: $\neg P_{1,3}$ (and $\neg P_{1,1}$ but already known) i.e. no pit in [2,2] or [1,3]
- Recall R_3 : $B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$ and R_5 : $B_{2,1}$
- Apply biconditional elimination, => elimination, and AND-elimination to R_3, followed by Modus Ponens with R_5 yields:
  R_{15}: $P_{1,1} \vee P_{2,2} \vee P_{3,1}$

i.e. there is a pit in [1,1], [2,2], or [3,1]

Proof by Resolution

- First application of the resolution rule: the literal $\neg P_{2,2}$ in R_{13} *resolves with* the literal $P_{2,2}$ in R_{15} to give the **resolvent** R_{16}: $P_{1,1} \vee P_{3,1}$

In English: if there's a pit in one of [1,1], [2,2], or [3,1] and it's not in [2,2], then it's in [1,1] or [3,1].

- Likewise, $\neg P_{1,1}$ in R_1 resolves with $P_{1,1}$ in R_{16} giving R_{17}: $P_{3,1}$

In English: if there's a pit in [1,1] or [3,1] and it's not in [1,1], then it's in [3,1].

- The last two inference steps are examples of the **unit resolution** inference rule: $\dfrac{l_1 \vee \cdots \vee l_k, \quad m}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k}$

Where each l is a literal and l_i and m are complementary literals (i.e. they negate each other).

Resolution

- A **clause** is a disjunction of literals.
- The unit resolution rule takes a **clause** and a literal and produces a new clause.
- The unit resolution rule can be generalized to the full **resolution** rule

$$\frac{l_1 \vee \cdots \vee l_k, \qquad m_1 \vee \cdots \vee m_n}{l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n}$$

where l_i and m_j are complementary literals.

- e.g., $\frac{P_{1,1} \vee P_{3,1}, \quad \neg P_{1,1} \vee \neg P_{2,2}}{P_{3,1} \vee \neg P_{2,2}}$
- The **resolution** inference rule is **sound** (Proof straightforward.)
- *A resolution-based theorem prover can, for any sentences KB and* α *in propositional logic, decide whether* $KB \models \alpha$ (**completeness of resolution**).

Conjunctive Normal Form

- The resolution rule applies only to clauses (i.e. disjunctions of literals).
- But completeness is ensured because: *Every sentence of propositional logic is logically equivalent to a conjunction of clauses.*
- A sentence expressed as a conjunction of clauses is said to be in **conjunctive normal form** or **CNF**.

Converting a sentence in propositional logic to CNF

- E.g. convert $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ into CNF:

1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

   $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$:

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$

Conjunctive Normal Form

3. To have only literals, we "move ¬ inwards" by repeated application of the following equivalences:

   $\neg(\neg\alpha) \equiv \alpha$ (double-negation elimination)

   $\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ (De Morgan)

   $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$ (De Morgan)

   By one application of the last rule we obtain:

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$

4. We distribute ∨ over ∧ wherever possible:

   $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

- The original sentence is now in CNF, as a conjunction of three clauses

A resolution algorithm

- Inference procedures based on resolution work by using the principle of **proof by contradiction**.
- To show that $KB \models \alpha$ we show that $(KB \wedge \neg \alpha)$ *is unsatisfiable.*
- This is done by proving a contradiction.

1. (KB $\wedge \neg \alpha$) is converted into CNF.
2. Resolution is applied to the resulting clauses: each pair that contains complementary literals is resolved to produce a new clause.
3. Add the new clause to the KB if it is not already there.
4. Continue the process until one of two things happens:
   - there are no new clauses that can be added, in which case KB does not entail α; or,
   - two clauses resolve to yield the *empty* clause, in which case KB entails α.

A resolution algorithm

- The empty clause—a disjunction of no disjuncts—is equivalent to *False* because a disjunction is true only if at least one of its disjuncts is true.
- An empty clause represents a contradiction since it arises only from resolving two complementary unit clauses such as P and ¬P.

Wumpus world:

- When the agent is in [1,1], there is no breeze, so there can be no pits in neighbouring squares; so:

  $KB = R_2 \wedge R_4 = (B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})) \wedge \neg B_{1,1}$
- We want to prove α which is, say, $\neg P_{1,2}$.
- When we convert $(KB \wedge \neg \alpha)$ into CNF, we obtain the clauses at the top of the following figure:

Wumpus resolution example

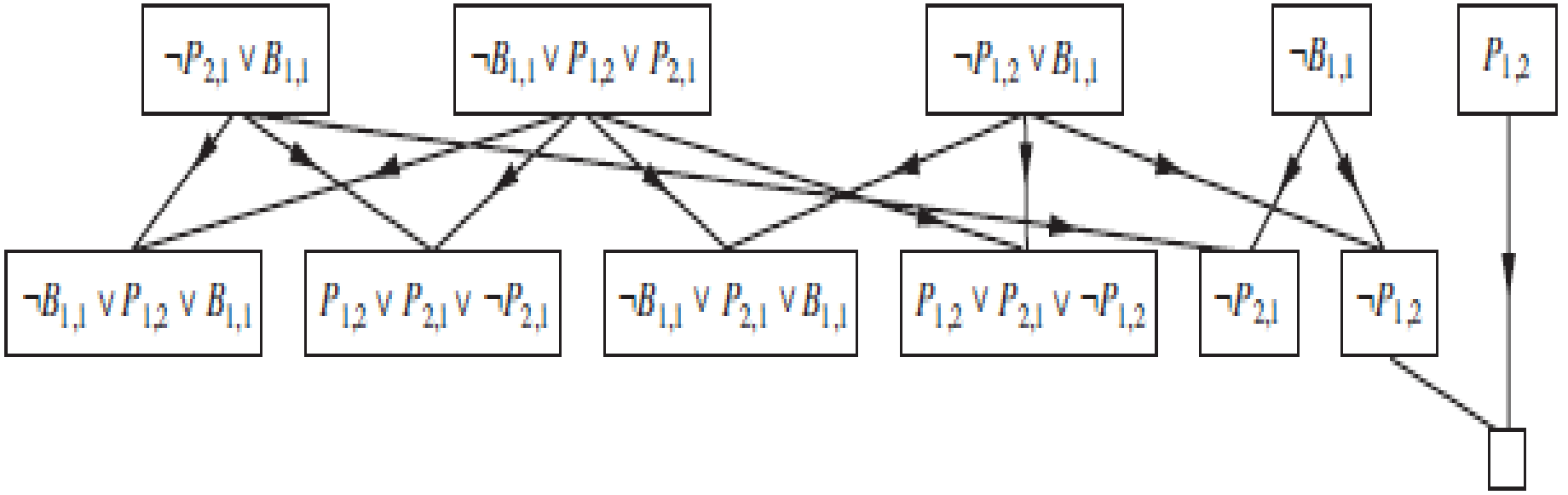


Figure 7.13 Partial application of PL-RESOLUTION to a simple inference in the wumpus world. $\neg P_{1,2}$ is shown to follow from the first four clauses in the top row.

Wumpus resolution example

The above resolution example could have been more efficient:

$\neg P_{1,2} \vee B_{1,1}$

$\neg B_{1,1}$

$P_{1,2}$

$\neg P_{1,2} \vee B_{1,1}$ $\neg B_{1,1}$

$\neg P_{1,2}$ $P_{1,2}$

[]

- Contradiction so $\neg P_{1,2}$ is true.
- Efficiency of resolution will be discussed.

Horn clauses and definite clauses

- Resolution is complete which makes it a very important inference method.
- However, in many practical situations, the full power of resolution is not needed.
- Certain restrictions on the form of sentences used in the KBs allow for the use of a more restricted and efficient inference algorithm.
- The **definite clause** is one such restricted form. It is a disjunction of literals of which *exactly one is positive*. E.g.
  - The clause $\neg L_{1,1} \vee \neg \text{Breeze} \vee B_{1,1}$ is a *definite clause*,
  - $\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$ is not a definite clause
- **Horn clause**: a disjunction of literals of which *at most one is positive* (Slightly more general than definite clause).
- All definite clauses are Horn clauses, as are clauses with no positive literals; these are called **goal clauses**.

Horn clauses and definite clauses

- Note that Horn clauses are closed under resolution: i.e. resolving two Horn clauses, produces a Horn clause.
- KBs of only definite clauses have advantages:
  - Every definite clause can be written as an implication whose premise is a conjunction of positive literals and whose conclusion is a single positive literal.
    - E.g. $\neg L_{1,1} \vee \neg \text{Breeze} \vee B_{1,1}$ can be written as the implication $L_{1,1} \wedge \text{Breeze} \Rightarrow B_{1,1}$
    - the premise is called the **body** and the conclusion is called the **head**.
    - A sentence consisting of a single positive literal, such as $L_{1,1}$, is called a **fact**. E.g. $L_{1,1}$ (equivalent to $\text{True} \Rightarrow L_{1,1}$)
  - Inference with Horn clauses can be done through the **forward chaining** and **backward chaining** algorithms which are the basis for **logic programming.**
  - Deciding entailment with Horn clauses can be done in **time that is *linear*** in the size of the knowledge base

A Grammar for CNF

$$
\begin{aligned}
\mathit{CNFSentence} &\rightarrow \mathit{Clause}_1 \wedge \cdots \wedge \mathit{Clause}_n \\
\mathit{Clause} &\rightarrow \mathit{Literal}_1 \vee \cdots \vee \mathit{Literal}_m \\
\mathit{Literal} &\rightarrow \mathit{Symbol} \mid \neg \mathit{Symbol} \\
\mathit{Symbol} &\rightarrow P \mid Q \mid R \mid \ldots \\
\mathit{HornClauseForm} &\rightarrow \mathit{DefiniteClauseForm} \mid \mathit{GoalClauseForm} \\
\mathit{DefiniteClauseForm} &\rightarrow (\mathit{Symbol}_1 \wedge \cdots \wedge \mathit{Symbol}_l) \Rightarrow \mathit{Symbol} \\
\mathit{GoalClauseForm} &\rightarrow (\mathit{Symbol}_1 \wedge \cdots \wedge \mathit{Symbol}_l) \Rightarrow \mathit{False}
\end{aligned}
$$

Figure 7.14 A grammar for conjunctive normal form, Horn clauses, and definite clauses. A clause such as $A \wedge B \Rightarrow C$ is still a definite clause when it is written as $\neg A \vee \neg B \vee C$, but only the former is considered the canonical form for definite clauses. One more class is the k-CNF sentence, which is a CNF sentence where each clause has at most k literals.

Forward Chaining Algorithm

- The **forward-chaining algorithm** PL-FC-ENTAILS?(KB, q) determines if a single proposition symbol q —the query— is entailed by a KB of definite clauses.
- It begins from known facts (positive literals) in the KB.
- If all the premises of an implication are known, then its conclusion is added to the set of known facts.

  E.g., if $L_{1,1}$ and *Breeze* are known and $(L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$ is in the KB, then $B_{1,1}$ can be added.
- This process continues until the query q is added or until no further inferences can be made.
- This forward chaining algorithm runs in linear time.
- It is easy to see that it is **sound**: every inference is essentially an application of Modus Ponens.
- Forward chaining is also **complete**: every entailed atomic sentence will be derived.

Forward Chaining Algorithm

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
          q, the query, a proposition symbol
  count ← a table, where count[c] is the number of symbols in c's premise
  inferred ← a table, where inferred[s] is initially false for all symbols
  agenda ← a queue of symbols, initially symbols known to be true in KB

  while agenda is not empty do
    p ← POP(agenda)
    if p = q then return true
    if inferred[p] = false then
      inferred[p] ← true
      for each clause c in KB where p is in c.PREMISE do
        decrement count[c]
        if count[c] = 0 then add c.CONCLUSION to agenda
  return false
```

Figure 7.15 The forward-chaining algorithm for propositional logic. The *agenda* keeps track of symbols known to be true but not yet "processed." The *count* table keeps track of how many premises of each implication are as yet unknown. Whenever a new symbol p from the agenda is processed, the count is reduced by one for each implication in whose premise p appears (easily identified in constant time with appropriate indexing.) If a count reaches zero, all the premises of the implication are known, so its conclusion can be added to the agenda. Finally, we need to keep track of which symbols have been processed; a symbol that is already in the set of inferred symbols need not be added to the agenda again. This avoids redundant work and prevents loops caused by implications such as $P \Rightarrow Q$ and $Q \Rightarrow P$.

Example with the Forward Chaining Algorithm

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

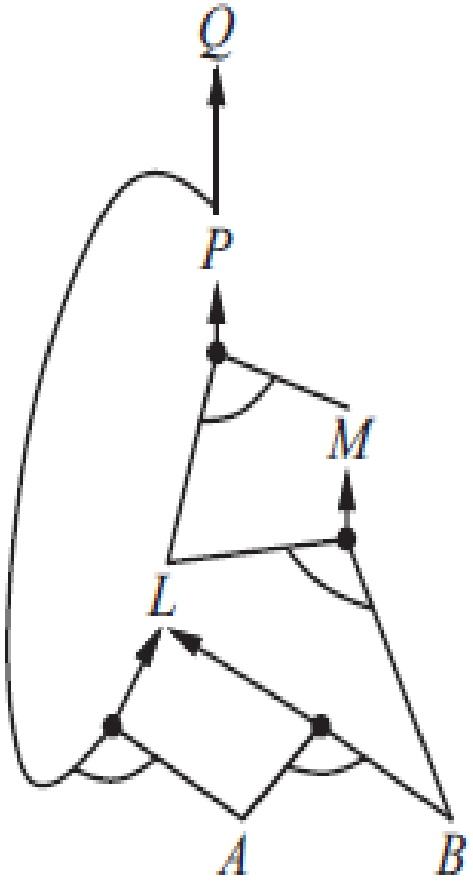
$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B

(a)



(b)

Figure 7.16 (a) A set of Horn clauses. (b) The corresponding AND–OR graph.

Forward Chaining

- Forward chaining is an example of the general concept of **data-driven** reasoning.
- In **data-driven** reasoning, the focus of attention starts with the known data.
- It can be used within an agent to derive conclusions from incoming percepts, often without a specific query in mind.
  - E.g., the wumpus agent might TELL its percepts to the KB using an incremental forward-chaining algorithm in which new facts can be added to the agenda to initiate new inferences.
- In humans, a certain amount of data-driven reasoning occurs as new information arrives.
  - E.g., if one is indoors and hears rain starting to fall, it might occur to him that the picnic will be canceled.

Backward Chaining

- The backward-chaining algorithm works backward from the query.
  - If the query *q* is known to be true, then no work is needed.
  - Otherwise, the algorithm finds those implications in the knowledge base whose conclusion is *q*.
  - If all the premises of one of those implications can be proved true (by backward chaining), then *q* is true.
- When applied to the query *Q* in Figure 7.16, it works back down the graph until it reaches a set of known facts, *A* and *B*, that forms the basis for a proof.

Example with the Backward Chaining Algorithm

$P \Rightarrow Q$

$L \wedge M \Rightarrow P$

$B \wedge L \Rightarrow M$

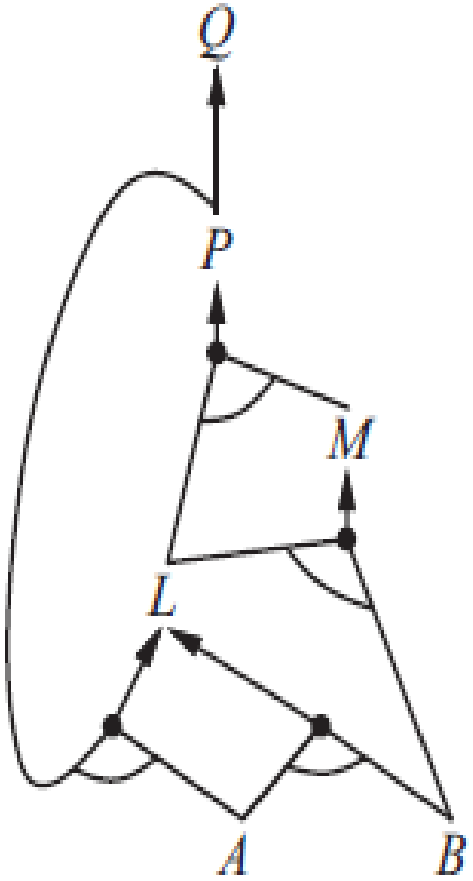
$A \wedge P \Rightarrow L$

$A \wedge B \Rightarrow L$

A

B

(a)



(b)

Figure 7.16 (a) A set of Horn clauses. (b) The corresponding AND–OR graph.

Backward Chaining

- The algorithm is essentially identical to the AND-OR-GRAPH-SEARCH algorithm in Figure 4.11.
- As with forward chaining, an efficient implementation of Backward Chaining runs in linear time.
- Backward chaining is a form of **goal-directed reasoning**. It is useful for answering specific questions such as
  - "Is the battery faulty?"
  - "Can I cook this dish?"
- Often, the cost of backward chaining is *much less* than linear in the size of the KB, because the process touches only relevant facts.

Agents Based on Propositional Logic

- Aim here is to construct a wumpus agent based on PL.
- **First step**: enable the agent to deduce all it can about the state of the world given its percept history.
- ➔ write down a complete logical model of the effects of actions.
- Consider the problem of deducing the current state of the wumpus world.
- We started with a large collection of sentences of the following form:

  $B_{1,1} \Leftrightarrow P_{1,2} \vee P_{2,1}$ $S_{1,1} \Leftrightarrow W_{1,2} \vee W_{2,1}$ ……..

- The agent knows there is exactly one wumpus, expressed in two parts. **First**, that there is *at least one* wumpus:

  $W_{1,1} \vee W_{1,2} \vee \ldots \vee W_{4,3} \vee W_{4,4}$

Agents Based on Propositional Logic

- Then, that there is *at most one* wumpus: i.e. For each pair of locations, we add a sentence that at least one of them must be wumpus-free:

  $\neg W_{1,1} \vee \neg W_{1,2}$

  $\neg W_{1,1} \vee \neg W_{1,3}$

  . . .

  $\neg W_{4,3} \vee \neg W_{4,4}$

- We associate time stamps to different percepts (*Stench, Breeze, etc.) to avoid contradictions* like ¬Stench in the KB, and then Stench in KB (some time later).
- This was supplied to MAKE-PERCEPT-SENTENCE in Fig. 7.1.
- So no problem having $Stench^4$ to the KB, rather than *Stench*, neatly avoiding any contradiction with $\neg Stench^3$.
- Symbols associated with permanent aspects of the world: no need for time superscript. Sometimes called **atemporal variables**.

Agents Based on Propositional Logic

- E.g., the initial KB includes

  $L_{1,1}^{0}$:the agent is in square [1, 1] at time 0

  FacingEast^0, HaveArrow^0, and WumpusAlive^0
- We use the word **fluent** (in the sense of flowing) to refer to an aspect of the world that changes.
- We can connect stench and breeze percepts directly to the properties of the squares where they are experienced through the location fluent as follows.

  For any time step t and any square [x, y], we assert:

  $L_{x,y}^{t} \Rightarrow (Breeze^{t} \Leftrightarrow B_{x,y})$ $\quad$ $L_{x,y}^{t} \Rightarrow (Stench^{t} \Leftrightarrow S_{x,y})$
- We need axioms that allow the agent to keep track of fluents such as $L_{x,y}^{t}$.
- We need to write down the **transition model** of the wumpus world as a set of logical sentences that capture fluents changes as the result of the agent's actions

Agents Based on Propositional Logic

- First, we need proposition symbols for the occurrences of actions. These will also be indexed by time
  - e.g. Forward^0 : the agent executes the *Forward* action at time 0.
- By convention, percept at time step *t* happens first, then action for time step *t*, then transition to time step *t+1*.
- Need to write **effect axioms** that specify the outcome of an action at the next time step.
- E.g. if the agent is at location [1,1] facing east at time 0 and goes Forward, the result is that the agent is in square [2,1] and no longer in [1,1]:

$$L^0_{1,1} \wedge \text{FacingEast}^0 \wedge \text{Forward}^0 \Rightarrow (L^1_{2,1} \wedge \neg L^1_{1,1})$$

- We need one such sentence for each possible time step, for each of the 16 squares, and each of the four orientations.
- We also need similar sentences for the other actions: *Grab*, *Shoot*, *Climb*, *TurnLeft*, and *TurnRight*.

The remaining slides are required reading.

Agents Based on Propositional Logic

- Suppose the agent decides to move *Forward* at time 0 and asserts this fact into its knowledge base.
- Given the previous effect axiom and the initial assertions about the state at time 0, the agent can deduce that it is in [2, 1] (at time 1).
- So ASK(KB,$L_{2,1}^{1}$) = *true. Good!*
- **Problem**: ASK(KB,$HaveArrow^{1}$) returns *false :* the agent cannot prove it STILL has the arrow. (It cannot prove it does not have it either!)
- **The reason**: the effect axiom fails to state what remains *unchanged* as the result of an action.
- The need to do this gives rise to the **frame problem**.

Agents Based on Propositional Logic

Possible solution to the frame problem: Add **frame axioms** explicitly asserting all the propositions that remain the same.

- e.g., for each time t we would have

$\text{Forward}^t \Rightarrow (\text{HaveArrow}^t \Leftrightarrow \text{HaveArrow}^{t+1})$

$\text{Forward}^t \Rightarrow (\text{WumpusAlive}^t \Leftrightarrow \text{WumpusAlive}^{t+1})$

. . .

Drawback:

- The proliferation of frame axioms is remarkably inefficient.
- In a world with *m* different actions and *n* fluents, the set of frame axioms will be of size *O(m n)*.
- This manifestation of the frame problem is sometimes called the **representational frame problem**.
- This problem is significant because the real world has very many fluents.

Agents Based on Propositional Logic

- Fortunately for us humans, each action typically changes just a small number *k* of the total fluents: the world exhibits **locality.**
- So need to define the transition model with a set of axioms of size *O(m k)* rather than size *O(mn)*.
- There is also an **inferential frame problem**: problem of projecting forward the results of a *t* step plan of action in time *O(k t)* rather than *O(n t)*.
- Solution to the problem: change one's focus from writing axioms about *actions* to writing axioms about *fluents*.
- For each fluent *F*, have an axiom that defines the truth value of F^{t+1} in terms of fluents (including *F* itself) at time *t* and the actions that may have occurred at time *t*.

Agents Based on Propositional Logic

- The truth value of F^{t+1} can be set in one of two ways:
  - either the action at time t causes *F* to be true at *t+1*, or
  - *F* was already true at time t and the action at time *t* does not cause it to be false.
- An axiom of this form is called a **successor-state axiom.** It has this schema:

  $F^{t+1} \Leftrightarrow \mathit{ActionCausesF}^t \vee (F^t \wedge \neg \mathit{ActionCausesNotF}^t)$
- One of the simplest successor-state axioms is the one for *HaveArrow*:

  $\text{HaveArrow}^{t+1} \Leftrightarrow (\text{HaveArrow}^t \wedge \neg \text{Shoot}^t)$
- More elaborate axioms for agent's location:

$$L_{1,1}^{t+1} \Leftrightarrow (L_{1,1}^t \wedge (\neg \text{Forward}^t \vee \text{Bump}^{t+1})) \vee (L_{1,2}^t \wedge (\text{South}^t \wedge \text{Forward}^t)) \vee (L_{2,1}^t \wedge (\text{West}^t \wedge \text{Forward}^t))$$

Agents Based on Propositional Logic

- Given
  - a complete set of successor-state axioms and
  - the other axioms listed at the beginning of this section,

the agent will be able to ASK and answer any answerable question about the current state of the world.

- E.g. initial sequence of percepts and axioms:

$\neg \text{Stench}^0 \wedge \neg\text{Breeze}^0 \wedge \neg\text{Glitter}^0 \wedge \neg\text{Bump}^0 \wedge \neg\text{Scream}^0$; Forward^0

$\neg\text{Stench}^1 \wedge \text{Breeze}^1 \wedge \neg\text{Glitter}^1 \wedge \neg\text{Bump}^1 \wedge \neg\text{Scream}^1$; TurnRight^1

$\neg\text{Stench}^2 \wedge \text{Breeze}^2 \wedge \neg\text{Glitter}^2 \wedge \neg\text{Bump}^2 \wedge \neg\text{Scream}^2$; TurnRight^2

$\neg\text{Stench}^3 \wedge \text{Breeze}^3 \wedge \neg\text{Glitter}^3 \wedge \neg\text{Bump}^3 \wedge \neg\text{Scream}^3$; Forward^3

$\neg\text{Stench}^4 \wedge \neg\text{Breeze}^4 \wedge \neg\text{Glitter}^4 \wedge \neg\text{Bump}^4 \wedge \neg\text{Scream}^4$; TurnRight^4

$\neg\text{Stench}^5 \wedge \neg\text{Breeze}^5 \wedge \neg\text{Glitter}^5 \wedge \neg\text{Bump}^5 \wedge \neg\text{Scream}^5$; Forward^5

$\text{Stench}^6 \wedge \neg\text{Breeze}^6 \wedge \neg\text{Glitter}^6 \wedge \neg\text{Bump}^6 \wedge \neg\text{Scream}^6$

Agents Based on Propositional Logic

- At this point, we have ASK(KB,$L^6_{1,2}$) = true ➔ the agent knows where it is.
- Also, ASK(KB,$W_{1,3}$) = true and ASK(KB,$P_{3,1}$) = true

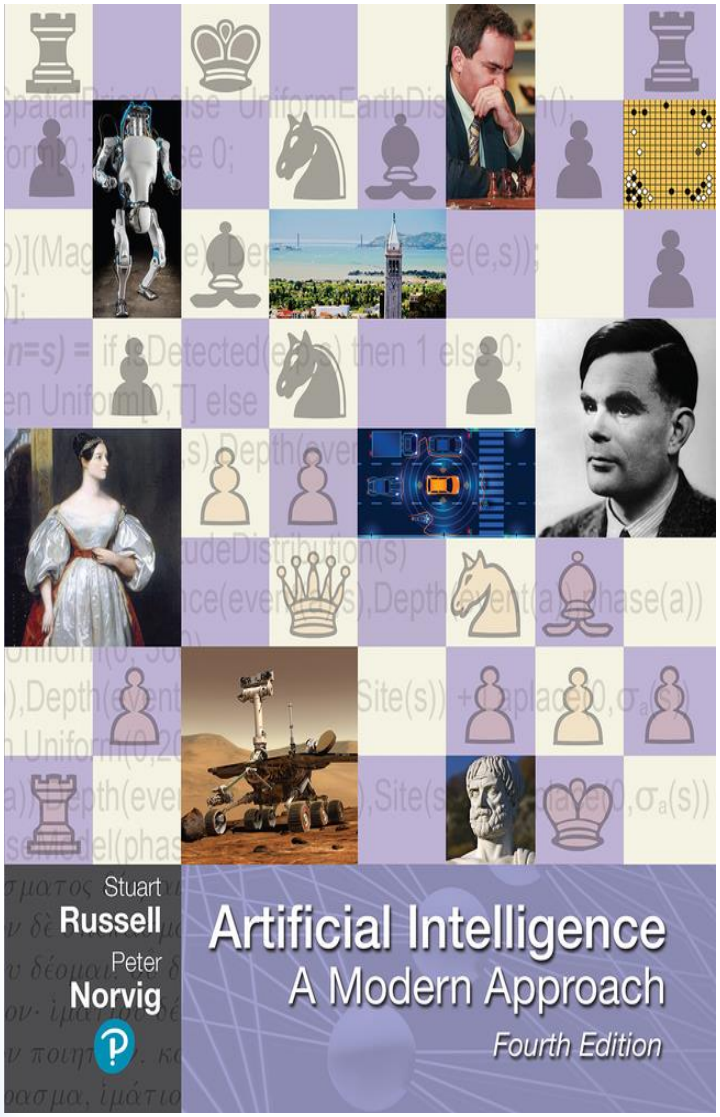
➔ the agent has found the wumpus and one of the pits.

- Asking whether a square is OK to move into (i.e. contains no pit nor live wumpus)?

$$OK^t_{x,y} \Leftrightarrow \neg P_{x,y} \wedge \neg(W_{x,y} \wedge \mathrm{WumpusAlive}^t)$$

- A serious problem remains: need to confirm that *all* necessary preconditions of an action hold for it to have its intended effect.
- BUT many unusual exceptions could cause an action to fail.
- Specifying all these exceptions is called the **qualification problem**.
- No complete solution within logic; left to the system designers' good judgment in deciding how detailed they want to be.

Slides based on the textbook



- Russel, S. and Norvig, P. (2020) Artificial Intelligence, A Modern Approach (4th Edition), Pearson Education Limited.